

Characters and Strings

Character handling libraries



Review: character vs. string

- Character a single element
 - Can be letter, number, special character
- String is a sequence of characters terminated by the null character '\0'
- Characters are the building blocks of strings



Operations on characters

- Character Input/Output
- Adding/subtracting characters
- Character manipulation
 - case conversion, searching, character replacement, etc.



Character Manipulation

- `#include<ctype.h>`

Function	Description
<code>int isalpha(int c)</code>	Returns non-zero if c is an alphabet
<code>int isdigit(int c)</code>	Returns non-zero if c is a digit
<code>int isalnum(int c)</code>	Returns non-zero if c is alpha numeric
<code>int isspace(int c)</code>	Returns non-zero if c is a white space(CR, FF, HT, NL, VT, space)
<code>int tolower(int c)</code>	Returns lower case value of c
<code>int toupper(int c)</code>	Returns upper case value of c



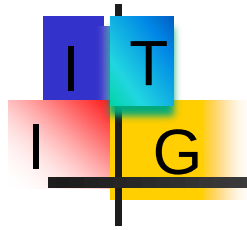
Example

- Write a function to count the number of characters and words in a string?

```
void WordsnChars(const char *str, int *words, int *chars)
```

Input: Hello World <CR>

Output: Words:2, Characters: 10



Handling Strings

Standard Input/Output



String variables

- Allocate an array of a size large enough to hold the string (plus 1 extra value for the delimiter)

```
char str[12]; // String variable  
char str[12] = "Hello World";
```

```
char *str1 = "hello world";
```



String Input

- Formatted input specifier %s
 - ignores leading white space
 - reads characters until next white space encountered
 - stores null (\0) char after last non-white space char
 - Reads into array
- Example

```
char name[11];
scanf("%s",name);
```

 - Buffer overflow if number of characters more than array size

How to prevent this

Option 1: Width-limited scanf You can tell scanf the maximum number of characters to read. Since the buffer is 11, the max string is 10.

scanf("%10s", name); // Reads a maximum of 10 characters

Option 2: Use fgets (Better) The fgets function is much safer because it requires you to specify the buffer size.

fgets(name, 11, stdin); // Reads at most 11 bytes from input



Reading until newline

- `char *gets(char *str)`
 - return `str` or `NULL` if problem
 - reads until `\n` encountered
 - does not allow to limit the number of characters read (may result in buffer over flow)
 - `gets` is **dangerous** and may not be used

Eg: `char name[10];`
`gets(name);`



String output

- Use %s field specification in printf
characters in string printed until \0 encountered



Printing string using puts

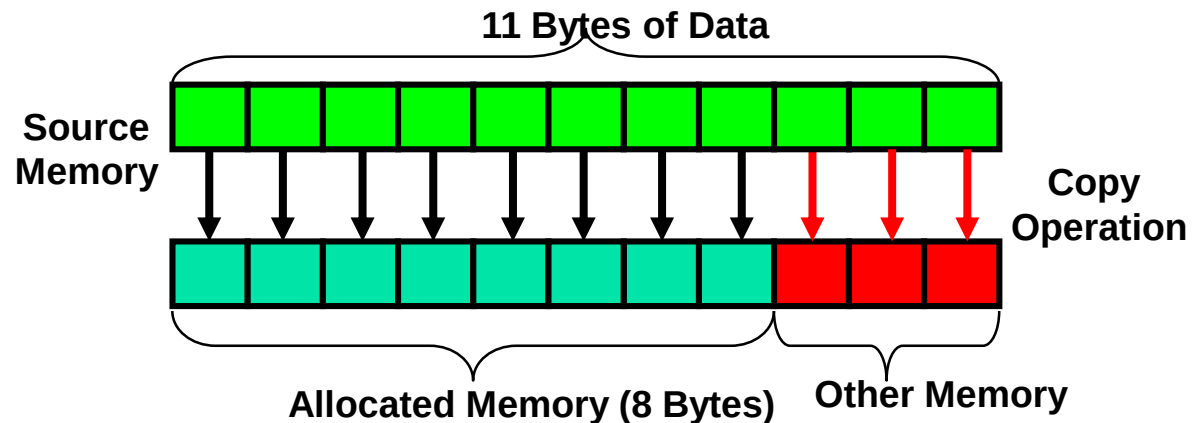
- `int puts(char *str)`
 - prints the string pointed to by `str` to standard output
 - prints until delimiter reached (string better have a `\0`)
 - returns EOF if the puts fails

Eg: `puts(name);`

puts() automatically adds a newline character (`\n`) to the end of the output.

Buffer overflow

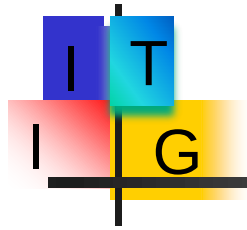
- A buffer overflow occurs when data is written outside of the boundaries of the memory allocated to a particular data structure.





Buffer overflow - continued

- Buffer overflow occur because we usually do not check bounds.
 - Standard library functions do not check bounds.
 - Programmers do not check bounds.
- Not all buffer overflows are exploitable.



Handling Strings

String manipulation functions



String length

- `size_t strlen(char const *)`
 - Returns the number of characters in a string excluding the NULL character.
 - `size_t` is unsigned long type, used to define sizes of strings and (other) memory blocks
 - Include `string.h`

```
char *str2 = "Hello World";  
printf("%lu", strlen(str2));
```

```
#include <stdio.h>  
#include <string.h>  
int main(){  
    char *str = "Hello world";  
    printf("%d", strlen(str));  
    return 0;  
}
```



Example: String length

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[20] = "Programming";
    printf("Length of string str1: %lu", strlen(str1));
    return 0;
}
```

How is it different from sizeof()?



*strlen() tells you about the content of the string, while sizeof() tells you about the container's memory size.
If you ran sizeof(str1), the result would be 20.*



String length limited to maxlen

- `size_t strlen(const char *str, size_t maxlen)`
 - Returns length of string if it is less than *maxlen* else returns *maxlen*

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[20] = "Programming";
    printf("Length of string str1 : %lu", strlen(str1, 30));
    printf("Length of string str1 : %lu", strlen(str1, 7));
    return 0;
}
```



String copy

- `char *strcpy(char *dst, char const *src);`
 - Copy string at src to dst
 - Programmer's responsibility to ensure that destination region large enough to hold result
- Include `string.h`



String copy

- `char *strcpy(char *str1, char *str2)`
 - Copies string `str2` into string `str1`, including null character

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[30] = "Hello";
    char s2[30] = "Computer Programming";
    strcpy(s1,s2);
    printf("String s1 is %s",s1);
    return 0;
}
```

*String s1 is Computer Programming
'Hello' is replaced by the 'Computer Programming'*



String copy of length n

- `char *strncpy(char *dst, char *src, size_t n)`
 - If `src > n` copies first `n` characters of `src` into `dst`
 - If `src < n`, copies entire `src` into `dst`, and appends several “\0” characters to make length of `dst`

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[30] = "Hello";
    char s2[30] = "Computer Programming";
    strncpy(s1,s2, 12);
    printf("String s1 is %s",s1);
    return 0;
}
```

It copies only 12 characters of s2 in s1



String concatenation

- `char *strcat(char *dst, char *src)`

- Concatenates two strings and returns the concatenated string

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[10] = "Hello";
    char s2[10] = "World";
    strcat(s1,s2);
    printf("Output %s",s1);
    return 0;
}
```

*strcat finds the `\0` at the end of `s1`.
It starts copying "World" from that position.*



String concatenation limited to maxlen

- `char *strncat(char *dst, char *src, int n)`

- Concatenates `n` characters of `str2` to `str1`. Null character appended at the end of the string

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[10] = "Hello";
    char s2[10] = "World";
    strncat(s1,s2,3);
    printf("Output %s",s1);
    return 0;
}
```

*strncat finds the \0 at the end of s1.
It starts copying 3 character of "World" from that position.*